

4.26

1.Transformer 简介

1.是什么

Transformer 是一种完全基于 自注意力机制 (Self-Attention) 的神经网络结构。

它抛弃了此前主流的循环网络 (RNN、LSTM) 和卷积网络 (CNN) 的串行或局部处理方式，改为让序列中所有位置两两直接交互，从而一次性地建模全局依赖关系。

核心组件包括：

- 自注意力：让每个词在编码时都能“看到”整个句子，并根据与所有词的相关度生成新的表示。
- 多头注意力：用多组不同的注意力，让模型从不同角度捕捉语义。
- 位置编码：因为没有了循环和卷积，需要显式注入位置信息，使模型知道词的先后顺序。
- 编码器-解码器结构 (原始设计)：编码器理解输入，解码器生成输出。

2.用来做什么

自然语言处理 (机器翻译)、计算机视觉、多模态等。

3.比原先技术的优势

并行计算能力，训练速度极快

- 旧技术：RNN/LSTM 必须按时间步串行处理，第 t 步依赖第 $t-1$ 步的隐藏状态，无法并行。长序列训练非常慢，且难以充分利用 GPU。
- Transformer：自注意力一次性计算整个序列中所有位置的关系，训练时完全并行，训练效率提高，从而支持海量数据和大规模模型。

完美解决距离依赖问题

- 旧技术：RNN 随着序列增长，早期信息容易“遗忘”。RNN 天然带有邻近优先的归纳偏差，对近处关系敏感，但可能忽略远处信号。CNN 受限于卷积核感受野，堆叠多层才能看到全局。
- Transformer：任意位置无论多远，模型都能直接捕捉依赖（例如第 1 个词和第 100 个词的关系），这对理解长文本、生成连贯段落至关重要。

2.Transformer 具体结构及训练步骤

本代码是将德语翻译为英语的实现。

具体结构

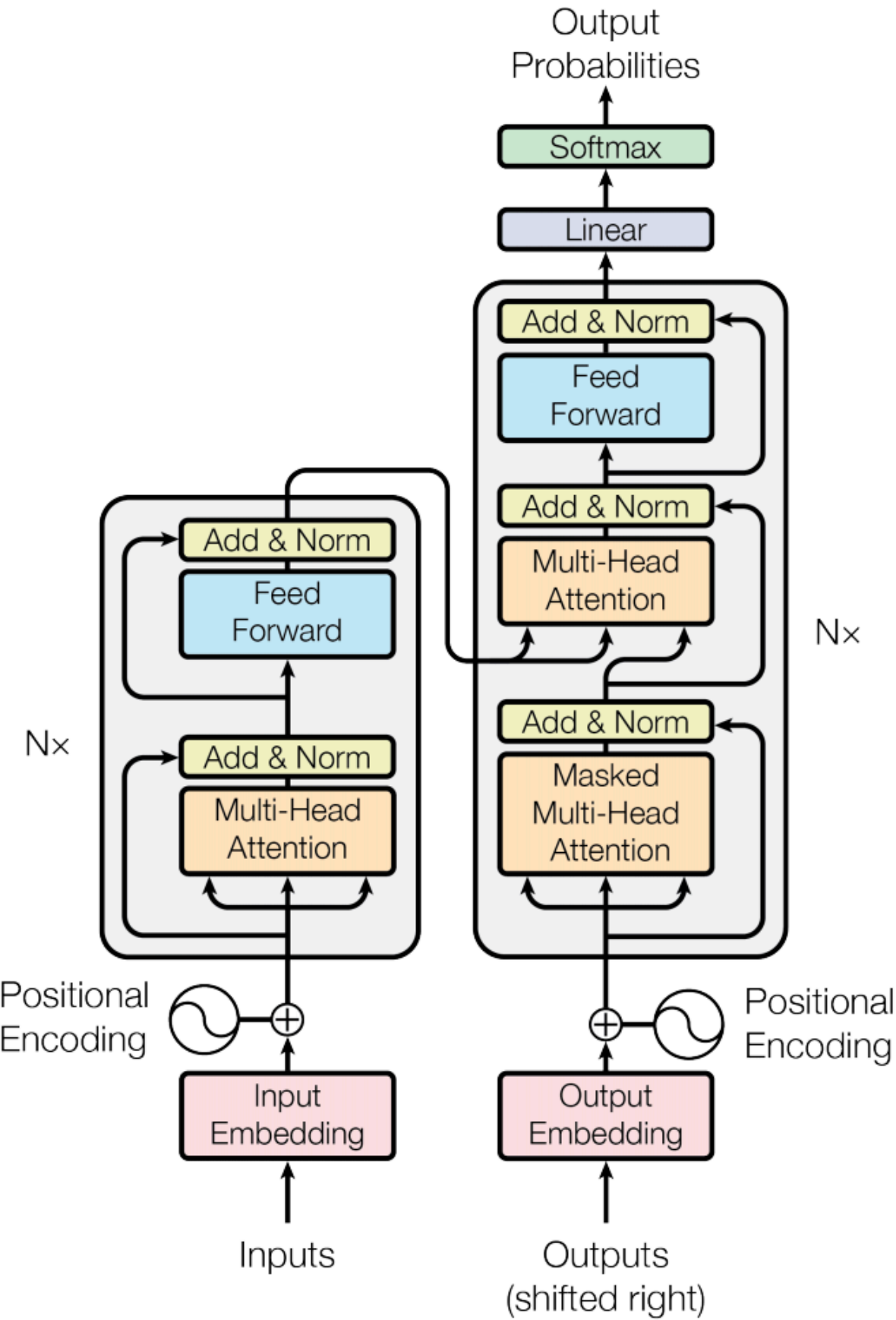


图2.1 Transformer 结构

训练过程，首先介绍一下 Encode 部分，也就是上图左边的一小块儿。

数据预处理

处理需要翻译的原句、预期结果，将一句话按照空格和标点分割，一个词就是一个 token，按照自己设定好的字典转化成对应的数字，变成一组向量。

例：'ich mochte ein bier P' (德语) 转变成 [1, 2, 3, 4, 0]。

例子中的 P 意思是 Padding，对于两个长短不一的句子，把短的句子填充上 padding，让两个张量形状一样，便于处理。

Embedding

原词嵌入、目标词嵌入。

把预处理之后的这组向量的每个元素经过线性变换再变成一组数据，原论文和代码实现一样都是512维。

现在上面举的例子变成了5x512的矩阵。

线性变换所用到的矩阵是模型的参数，首轮训练时候随机生成，之后反向传播每次更新。

位置编码

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

图2.2 位置编码所采用的函数

观察上面的函数，位置编码仅需要 pos (每个 token 的位置索引)、i (嵌入维度的索引)。

这两个变量和模型参数没有关系，所以这一步用不到模型参数，反向传播也不需要更新这个。

位置编码计算完之后，直接加到嵌入向量上。

多头注意力

(这一部分还使用了残差链接)。

上一步计算之后得到的嵌入向量成为这一步的 inputs。

将 inputs 通过线性变换，乘以模型参数 W_Q ，并且分成多个头，作为 Q。

将 inputs 通过线性变换，乘以模型参数 W_K ，并且分成多个头，作为 K。

将 inputs 通过线性变换，乘以模型参数 W_V ，并且分成多个头，作为 V。(这三个 inputs 是一样的都是最开始那个)

接着，生成了一些掩码，这一步具体用于让模型忽略掉自己在数据预处理时候填充的 padding。

接下来调用了缩放点注意力。

缩放点注意力

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

图2.3 缩放点注意力

除以一个 d_k 的平方根，防止内积过大，加速收敛。

做完之后，多个头都拼了回来。

前馈神经网络

再经过一个简单的前馈神经网络。

```
# ===== 前馈网络 =====  
# 对应模型图中的 Feed Forward 和 Add & Norm  
class PositionwiseFeedForward(nn.Module):  
    def __init__(self):  
        super(PositionwiseFeedForward, self).__init__()  
        # 就是一个 MLP (两层全连接, 中间 ReLU)  
        self.fc = nn.Sequential(  
            nn.Linear(d_model, d_ff),      # 第一层: d_model -> d_ff  
            nn.ReLU(),                     # 激活函数  
            nn.Linear(d_ff, d_model)       # 第二层: d_ff -> d_model  
        )
```

图2.4 code

上述过程叠加好几层编码器之后就是 Encode 部分，接下来是 Decode 部分，也就是上面那个结构的右面一大块。

具体和上面有一些差别。

Tips：Decode 部分的输入是期望的翻译结果，Encode 部分的输入是原句子。

首先是交叉注意力，将 encode 部分的输出做为计算 K 和 V 的 inputs，计算 Q 时采用 decode 部分的 inputs。

因为 Q 和 K 会做向量乘积，这样就可以将原句子和目标句子联系起来。

又因为翻译时候逐词翻译，再结合上述的乘积过程，可以发现 Q 与 K 乘积的矩阵在 $j > i$ 时没有意义，所以通过掩码屏蔽掉。

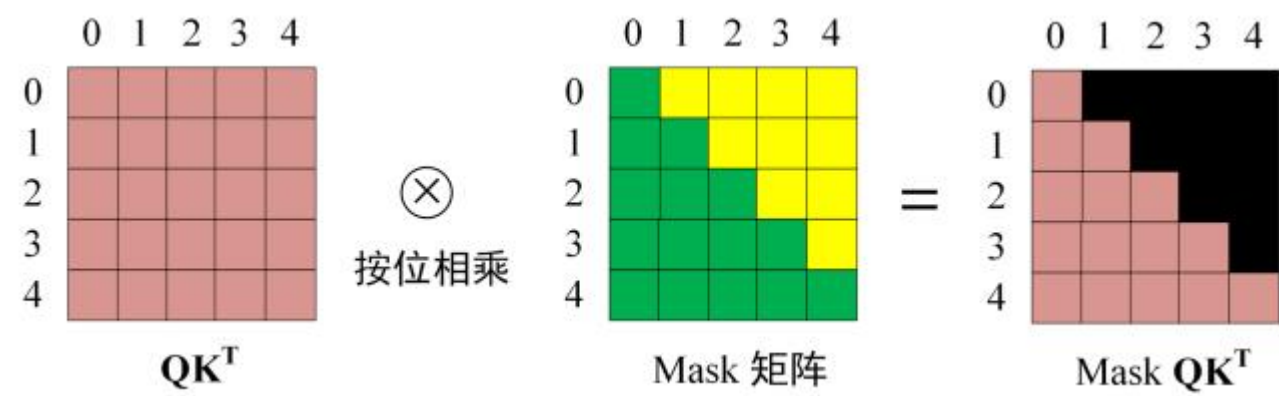


图2.5 掩码机制

其他区别不太大，最后得到的结果，经过一些投影变换，可以得到预测翻译单词的概率。

3.围绕 Transformer 的小心思

样本的问题

```
sentence = [  
    # 每一行是一个样本，包含三个字符串，空格分隔：  
    # enc_input 原始输入的源语言句子（德语），末尾用'P'填充至统一长度  
    # dec_input 解码器输入的目标语言句子，开头加上'S'起始符  
    # dec_output 解码器的期望输出目标语言句子，末尾加上'E'结束符  
    ['ich mochte ein bier P','S i want a beer .', 'i want a beer . E'],  
    ['ich mochte ein cola P','S i want a coke .', 'i want a coke . E'],  
]
```

图3.1 code

⚽ 这是测试时候的样例，观察 dec_input、dec_out，这两句话很像，那能不能一样呢？

答案是不能。

两个角度：

- 如果一样，模型在预测结果时候更倾向于复制输入的句子，因为训练时候输入输出一样，机智的 Transformer 会直接学到这个复制过程。
- 模型预测需要 S 标识符这个起始和 E 这个标识符作为结尾。

Q、K、V 实际意义的思考

从代码来看，我很难直接看出具体含义，论文给 Q、K、V 人为赋予了查询、键、值的含义。

根据缩放点注意力的函数，Q 和 K 做的是乘积，两者实际上可交换甚至是等价的，我猜只需要修改掩码部分的逻辑就可以实现 Q 和 K 的完全互换。

以上思考想要进行过一定改代码的尝试，时间受限最终没什么结果。